Java on the Command Line

Senior Level: Inside the Launcher, Security and Delivery

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

1. Inside the java Launcher
2. Class Loading
3. Class Files and Versions
4. Start-up and Profiling
5. Trust and Supply Chain
6. Delivery Beyond the Laptop
7. Remote Debugging and Diagnostics
8. Summary

Inside the java Launcher

What happens when you type `java -cp out App`

Algorithm 1: The `java` launcher (simplified)

Input: command line c, environment e

1 insert e[`JDK_JAVA_OPTIONS`] before the options of c; expand every `@file`;
2 split c into JVM options, the main class (or `-jar` / `-m`) and the program arguments;
3 load the JVM library (`libjvm.so`, `jvm.dll`, `libjvm.dylib`) and create the JVM;
4 the JVM reads e[`JAVA_TOOL_OPTIONS`], maps the CDS archive, resolves the boot modules;
5 **if** *-jar f* **then**
6 | *main* $\leftarrow$ `Main-Class` of f; class path $\leftarrow$ f + its `Class-Path`
7 **else**
8 | *main* $\leftarrow$ the given class
9 load *main* with the **application class loader**; link, verify, initialize;
10 call `main(String[])` on a new thread `main`;
11 wait for all non-daemon threads; run shutdown hooks; **return** *exit status*;

Where the JDK lives

Windows

```
where java
echo %JAVA_HOME%
dir "C:\Program Files\Eclipse Adoptium"
```

Installers add `bin` to the `PATH` and may set `JAVA_HOME`; the *first* `java.exe` on the `PATH` wins (beware of old `C:\ProgramData\Oracle\Java\javapath`).

macOS

```
/usr/libexec/java_home -V
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
ls /Library/Java/JavaVirtualMachines
```

`/usr/bin/java` is a stub that starts the JDK chosen by `java_home`. JDKs live in `.../jdk-21.jdk/Contents/Home`. Several versions side by side: `JAVA_HOME` selects one; tools such as SDKMAN! or jenv switch per project.

Argument files and option variables

codeoutput/ArgFiles/options.txt text

```
1 -cp out
2 -Dgreeting="Merhaba Dünya"
```

codeoutput/ArgFiles/App.java text

```
1 public class App {
2     public static void main(String[] args) {
3         String who = System.getProperty("greeting
          ", "Hello");
4         System.out.println(who + ", args = " +
          String.join(" ", args));
5     }
6 }
```

`JAVA_TOOL_OPTIONS` is read by **every** JVM, also the one inside `javac`; `JDK_JAVA_OPTIONS` only by the `java` launcher. Both are announced, so a stray value cannot hide. `@file` keeps long command lines (Windows limit: 8191 characters in `cmd`) out of scripts.

Argument files and option variables

codeoutput/ArgFiles/options.txt text

```
1 -cp out
2 -Dgreeting="Merhaba Dünya"
```

codeoutput/ArgFiles/App.java text

```
1 public class App {
2     public static void main(String[] args) {
3         String who = System.getProperty("greeting
          ", "Hello");
4         System.out.println(who + ", args = " +
          String.join(" ", args));
5     }
6 }
```

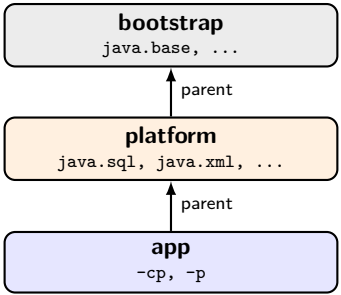
Terminal codeoutput/ArgFiles.txt

```
$ cat options.txt
-cp out
-Dgreeting="Merhaba Dünya"
$ java @options.txt App one two
Merhaba Dünya, args = one two
$ export JAVA_TOOL_OPTIONS=-Dgreeting=Hi
$ java -cp out App
Picked up JAVA_TOOL_OPTIONS: -Dgreeting=Hi
Hi, args =
$ javac -version
Picked up JAVA_TOOL_OPTIONS: -Dgreeting=Hi
javac 21.0.12.1
$ unset JAVA_TOOL_OPTIONS
$ export JDK_JAVA_OPTIONS=-Dgreeting=Selam
$ java -cp out App
NOTE: Picked up JDK_JAVA_OPTIONS: -Dgreeting=Selam
Selam, args =
$ javac -version
javac 21.0.12.1
```

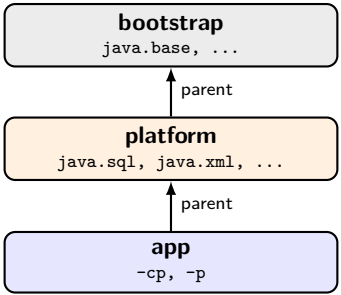
`JAVA_TOOL_OPTIONS` is read by **every** JVM, also the one inside `javac`; `JDK_JAVA_OPTIONS` only by the `java` launcher. Both are announced, so a stray value cannot hide. `@file` keeps long command lines (Windows limit: 8191 characters in `cmd`) out of scripts.

Class Loading

Three class loaders



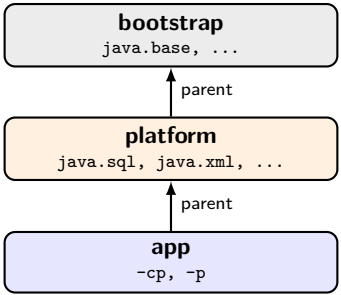
Three class loaders



Terminal codeoutput/Loaders.txt

```
$ java -cp out Loaders
Loaders  loaded by app
Driver   loaded by platform
String   loaded by bootstrap
  chain: app
  chain: platform
```

Three class loaders



Terminal codeoutput/Loaders.txt

```
$ java -cp out Loaders
Loaders  loaded by app
Driver   loaded by platform
String   loaded by bootstrap
  chain: app
  chain: platform
```

- A loader first asks its **parent**: a class path JAR cannot replace `java.lang.String`.
- The same name loaded by two loaders gives two *different* classes (plugins, application servers).

Watching the JVM load classes: `-Xlog`

Terminal codeoutput/ClassLoad.txt

```
$ java -Xlog:class+load -cp out App | wc -l
549
$ java -Xlog:class+load -cp out App | grep -c "shared objects file"
531
$ java -Xlog:class+load -cp out App | grep " App "
[0.028s][info][class,load] App source: file:~/project/out/
$ java -Xshare:off -Xlog:class+load -cp out App | grep -c "shared objects file"
0
```

About 550 classes are loaded to print one line; almost all come from the **CDS archive** (*shared objects file*), a memory image of pre-parsed JDK classes. `-Xlog` is the unified logging of the JVM: `-Xlog:help` lists the tags (`gc`, `class+load`, `safepoint`, ...).

Class Files and Versions

Inside a `.class` file

bytes	content	
0–3	magic	`CA FE BA BE`
4–5	minor version	`00 00`
6–7	major version	`00 41` = 65
8–	constant pool, …	

$$major = release + 44$$

so Java 8 → 52, 17 → 61, 21 → 65, 25 → 69.

Terminal `codeoutput/Version.txt`

```
$ javac Hello.java
$ od -A d -t x1 -N 8 Hello.class
0000000 ca fe ba be 00 00 00 41
0000008
$ javap -v Hello.class | grep major
  major version: 65
$ printf '\x00\x46' | dd of=Hello.class bs=1 seek=6 conv=notrunc 2>/dev/null
$ od -A d -t x1 -N 8 Hello.class
0000000 ca fe ba be 00 00 00 46
0000008
$ java Hello
Error: LinkageError occurred while loading main class Hello
        java.lang.UnsupportedClassVersionError: Hello has been compiled by
        a more recent version of the Java Runtime (class file version 70.0)
        , this version of the Java Runtime only recognizes class file
        versions up to 65.0
```

We changed byte 7 to 46 (= 70, Java 26): a JVM refuses class files **newer** than itself, but runs all older ones. Fix: run with a newer JDK, or compile with `-release 21`.

Start-up and Profiling

Faster start-up with an application CDS archive

Terminal codeoutput/Cds.txt

```
$ java -jar app.jar
2026-10-13: JLINK, JPACKAGE
$ time (for i in 1 2 3 4 5; do java -Xshare:off -jar app.jar > /dev/null; done)
0.739 s
$ time (for i in 1 2 3 4 5; do java -jar app.jar > /dev/null; done)
0.427 s
$ java -XX:ArchiveClassesAtExit=app.jsa -jar app.jar > /dev/null
$ ls -lh app.jsa | awk '{print $5, $9}'
144K app.jsa
$ time (for i in 1 2 3 4 5; do java -XX:SharedArchiveFile=app.jsa -jar app.jar > /dev/null; done)
0.428 s
```

Five runs each (times differ from machine to machine). `-Xshare:off` disables the JDK's archive; `-XX:ArchiveClassesAtExit` (Java 13+) records the classes of one run, `-XX:SharedArchiveFile` maps them next time. Projects Leyden (Java 24+: `-XX:AOTCache`) and GraalVM `native-image` go further.

Profiling from the command line: JFR

codeoutput/Jfr/Work.java **Java**

```java
import java.util.HashMap;
import java.util.Map;

public class Work {
    public static void main(String[] args) {
        Map<Integer, String> m = new HashMap<>();
        long sum = 0;
        for (int round = 0; round < 40; round++) {
            for (int i = 0; i < 200_000; i++)
                m.put(i, "v" + i);
          // allocates a lot
            sum += m.get(round).length();
            m.clear();
        }
        System.out.println("sum = " + sum);
    }
}
```

The **JDK Flight Recorder** is built into the JVM with about 1 % overhead, so it can stay on in production. `jcmd <pid> JFR.start` attaches it to a running JVM; JDK Mission Control (a separate GUI) opens the `.jfr` file.

Profiling from the command line: JFR

codeoutput/Jfr/Work.java — Java

```java
import java.util.HashMap;
import java.util.Map;

public class Work {
    public static void main(String[] args) {
        Map<Integer, String> m = new HashMap<>();
        long sum = 0;
        for (int round = 0; round < 40; round++) {
            for (int i = 0; i < 200_000; i++)
                m.put(i, "v" + i);
          // allocates a lot
            sum += m.get(round).length();
            m.clear();
        }
        System.out.println("sum = " + sum);
    }
}
```

Terminal — codeoutput/Jfr.txt

```
$ java -XX:StartFlightRecording=filename=rec.jfr -cp out Work
[0.520s][info][jfr,startup] Started recording 1. No limit specified, using
       maxsize=250MB as default.
[0.520s][info][jfr,startup]
[0.520s][info][jfr,startup] Use jcmd 9467 JFR.dump name=1 to copy recording
       data to file.
sum = 110
$ jfr summary rec.jfr | grep -E "Duration|jdk.(GarbageCollection|
       ExecutionSample|ObjectAllocationSample) "
 Duration: 1 s
 jdk.ObjectAllocationSample              120          1685
 jdk.ExecutionSample                      24           240
 jdk.GarbageCollection                     8           184
$ jfr print --events jdk.ExecutionSample rec.jfr | grep -m 3 "line:"
    java.util.HashMap.resize() line: 741
    java.util.HashMap.putVal(int, Object, Object, boolean, boolean) line: 669
    java.util.HashMap.put(Object, Object) line: 618
```

The **JDK Flight Recorder** is built into the JVM with about 1 % overhead, so it can stay on in production. `jcmd <pid> JFR.start` attaches it to a running JVM; JDK Mission Control (a separate GUI) opens the `.jfr` file.

Trust and Supply Chain

Same sources, same JAR? `jar -date`

Terminal codeoutput/Reproducible.txt

```
$ jar --create --file a.jar -C out .
$ sleep 2
$ jar --create --file b.jar -C out .
$ sha256sum a.jar b.jar | cut -c1-16,65-
81d7ce58a2c247a5  a.jar
aee817c341cfe5a4  b.jar
$ jar --create --date 2026-01-01T00:00:00Z --file a.jar -C out .
$ sleep 2
$ jar --create --date 2026-01-01T00:00:00Z --file b.jar -C out .
$ sha256sum a.jar b.jar | cut -c1-16,65-
76bfeafca1defab6  a.jar
76bfeafca1defab6  b.jar
```

A JAR is a ZIP file that stores each entry's time, so two builds differ. `-date` (Java 19+) fixes all timestamps: anyone can rebuild and compare checksums. Maven: `project.build.outputTimestamp`.

Checking a download

Terminal codeoutput/Checksum.txt

```
$ cat app.jar.sha256 | cut -c1-20,65-
b0c9fb42eac94d6a38b8  app.jar
$ sha256sum -c app.jar.sha256
app.jar: OK
$ sha256sum -c app.jar.sha256
app.jar: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

Every JDK download page publishes a SHA-256 checksum. Compare before installing:

Windows (Command Prompt)

```
certutil -hashfile OpenJDK21.msi SHA256
```

macOS (Terminal)

```
shasum -a 256 OpenJDK21.pkg
```

Signing and tampering

Terminal codeoutput/Signing.txt

```
$ keytool -genkeypair -alias haluk -keyalg RSA -keysize 3072 -validity 365 -dname "CN=Haluk Bingol, O=Yeditepe, C=TR" -keystore ks.p12 -storepass
        secret123
Generating 3,072 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=Haluk Bingol, O=Yeditepe, C=TR
$ jarsigner -keystore ks.p12 -storepass secret123 app.jar haluk 2>&1 | head -1
jar signed.
$ jar tf app.jar
META-INF/MANIFEST.MF
META-INF/HALUK.SF
META-INF/HALUK.RSA
META-INF/
App.class
$ jarsigner -verify app.jar 2>&1 | grep -v '^$' | head -1
jar verified.
$ cd out
$ jar uf ../app.jar App.class
$ cd ..
$ jarsigner -verify app.jar 2>&1 | grep -v '^$' | head -1
jarsigner: java.lang.SecurityException: SHA-384 digest error for App.class
$ java -jar app.jar 2>&1 | grep -m1 SecurityException
Exception in thread "main" java.lang.SecurityException: SHA-384 digest error for App.class
```

`jarsigner` stores a digest of every entry (`META-INF/*.SF`) and a signature (`*.RSA`). After we replaced `App.class`, verification and start-up both fail. Here the certificate is self-signed: real signing uses a certificate from a certificate authority. On **macOS** the *installer* must also be signed and notarized (`jpackage -mac-sign`).

Other risks on the command line

- **Class path order**: a JAR placed *before* yours can replace any of your classes; modules forbid split packages.
- `CLASSPATH` **and** `JAVA_TOOL_OPTIONS`: an attacker who can set the environment controls the JVM (`-javaagent:evil.jar`). Check them in CI logs: the JVM prints *Picked up* ....
- `PATH`: which `java` runs? (`where java`, `which -a java`)
- **Old JDKs**: only the latest update of an LTS gets security fixes (21.0.*x*); update monthly or with each quarterly release.
- **Security Manager**: deprecated in Java 17 and permanently disabled in Java 24; isolate untrusted code with operating system processes or containers instead.
- **Secrets**: never on the command line (`-Dpassword=...` is visible in `ps` and `jcmd VM.command_line`).

Delivery Beyond the Laptop

A container image with a jlink runtime

```
# ---- stage 1: build with a full JDK
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY src src
RUN javac -d mods --module-source-path src -m com.greet,com.app \
 && jlink -p mods --add-modules com.app --strip-debug \
          --no-header-files --no-man-pages --output /rt

# ---- stage 2: only the runtime and the application
FROM debian:bookworm-slim
COPY --from=build /rt /opt/app
USER 1000
ENTRYPOINT ["/opt/app/bin/java", "-m", "com.app/com.app.Main"]
```

```
docker build -t hello .
docker run --rm hello
```

Same commands on **Windows** and **macOS** with Docker Desktop. The JVM sees the container's CPU and memory limits; tune with `-XX:MaxRAMPercentage=75`. Not run here.

Native executables: GraalVM `native-image`

```
javac -d out App.java
native-image -cp out App app
./app            # starts in milliseconds
```

Not run here (needs GraalVM). On **Windows** it needs the Visual Studio C++ build tools and makes `app.exe`; on **macOS** the Xcode command line tools.

	JVM	native image
start-up	50–500 ms	1–50 ms
peak speed	JIT, very high	good
build time	seconds	minutes
reflection	free	must be declared
one binary for	every OS	one OS + CPU

Remote Debugging and Diagnostics

Diagnostics without an IDE

need	command
wait for a debugger	`java -agentlib:jdwp=transport=dt_socket,server=y,suspend=y,address=*:5005 -jar app.jar`
debug in the terminal	`jdb -attach 5005` (or attach an IDE to port 5005)
thread dump	`jcmd <pid> Thread.print`; macOS/Linux also `kill -3 <pid>`; **Windows** `Ctrl+Break`
heap dump	`jcmd <pid> GC.heap_dump heap.hprof`
dump on crash	`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=dumps`
garbage collector log	`-Xlog:gc*:file=gc.log:time,uptime`
record a profile	`-XX:StartFlightRecording=duration=60s,filename=r.jfr`
JVM settings of a process	`jcmd <pid> VM.flags, VM.system_properties, VM.command_line`

`address=*:5005` opens a port to the network: on a server use `localhost:5005` and an SSH tunnel.

Summary

Big ideas

- The `java` command is a small native launcher; options can come from the command line, `@files`, `JDK_JAVA_OPTIONS` and `JAVA_TOOL_OPTIONS`.
- Classes are loaded on demand by three loaders; CDS archives make start-up fast.
- A class file carries its version: newer class files do not run on older JVMs.
- Builds should be **reproducible**, downloads **checked**, artifacts **signed**.
- Delivery options grow from JAR to `jlink` image, installer, container and native executable; each one trades size, start-up and portability.
- The JDK's own tools (`jcmd`, `jfr`, `jdb`, `-Xlog`) diagnose a JVM anywhere a terminal exists.

Projects

1. Write `build.sh` and `build.bat` that build the books application reproducibly; prove it with two checksums on two different computers (one Windows, one macOS).
2. Measure start-up of the books application as a JAR, with AppCDS, as a `jlink` image and (optionally) as a native image. Report median times of 20 runs.
3. Sign the JAR, change one byte, and explain every line printed by `jarsigner -verify -verbose`.
4. Record a JFR file of `Work`, find the hottest method with `jfr print`, make it faster, record again.
5. Write a launcher script that picks the newest installed JDK ≥ 21 on Windows (`where`) and macOS (`java_home`) and prints a clear message if none exists.

Questions?